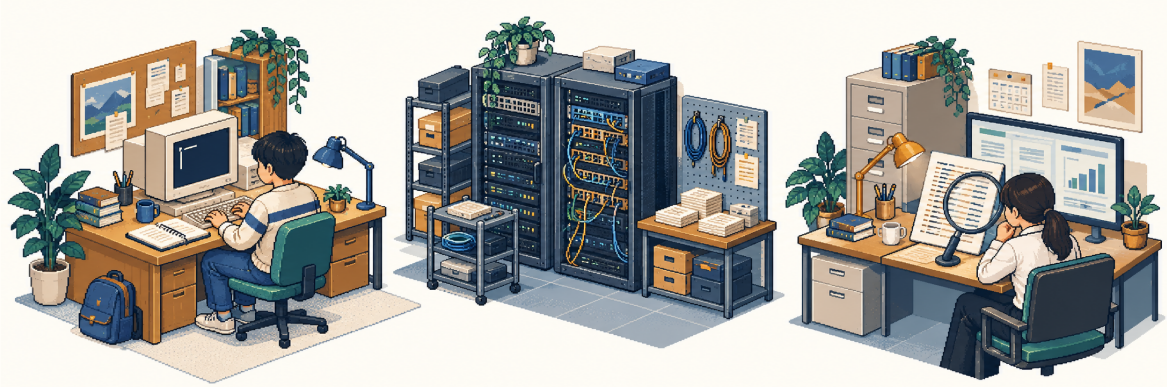
해킹·침해대응 실무 with Wazuh

시스템 침투와 SIEM 분석

SQL 인젝션 · SSH 인증 기록 · osquery 상태 조사

직접 실행하고, 기록을 찾고, 근거를 읽는다



학습자 → **실습 서버** → **Wazuh**

명령 한 개 실행 → 요청·상태 변화 기록 → 검색·원본 기록 확인

개념 삽화 · 생성형 이미지와 별도 조판한 설명. 실제 실행 화면이 아닙니다.

한 번에 한 동작을 실행하고, 그 결과를 읽으며 다음 단계로 이동합니다. 명령은 직접 입력하고 검색은 Wazuh의 Discover 화면에서 수행합니다.

> 이 시범본은 기존 강의의 실제 캡처를 활용한 편집 견본입니다. 설명 옆에 촬영 당시 화면과 새 절차의 차이를 표시했습니다. 새 절차의 전체 재실행·재촬영 전이므로 배포용 완성본은 아닙니다.

실습 범위: 설치된 kt66과 Wazuh 4.10.0을 사용하는 격리된 학습 환경. 이 문서의 공격은 강사가 지정한 대상에서만 수행합니다.

세 개의 작업 공간

이 책은 강사 또는 환경 준비 담당자가 kt66 구성을 완료하고 접속 정보를 제공한 상태에서 시작합니다. kt66은 공격 대상과 관제 도구를 함께 준비하는 실습 인프라입니다. Wazuh와 호스트 센서를 설치하는 과정은 건너뛰고, 준비된 기능을 확인하며 사용합니다.

같은 명령도 어디서 실행하느냐에 따라 다른 시스템을 바꿉니다. 터미널 제목에 아래 이름을 붙여 두세요.

• 공격자 터미널: 별도 Ubuntu 실습 PC. curl과 SSH 클라이언트를 사용합니다.

• 호스트 터미널 A·B: kt66이 설치된 Linux 호스트에 접속한 두 개의 터미널입니다. osquery 실습에서 사용합니다.

• 분석 브라우저: Wazuh Dashboard에 접속한 창입니다. 결과 검색은 이 창에서 합니다.

강사에게 호스트 주소, 웹 주소, Dashboard 계정, SSH 실습 계정과 대상 포트를 받으세요. 아래의 빈칸을 채운 뒤 진행합니다.

항목	내 환경
kt66 호스트 주소	_____
DVWA 웹 주소	_____
SSH 실습 계정 / 포트	_____ / 2204
Dashboard 주소	https://_____:5601

그림 속 192.168.0.189, 192.168.0.202, ccc는 촬영 환경의 값입니다. 자신의 환경과 같다고 가정하지 않습니다. 꺾쇠괄호로 표시한 자리는 실제 값으로 바꿉니다.

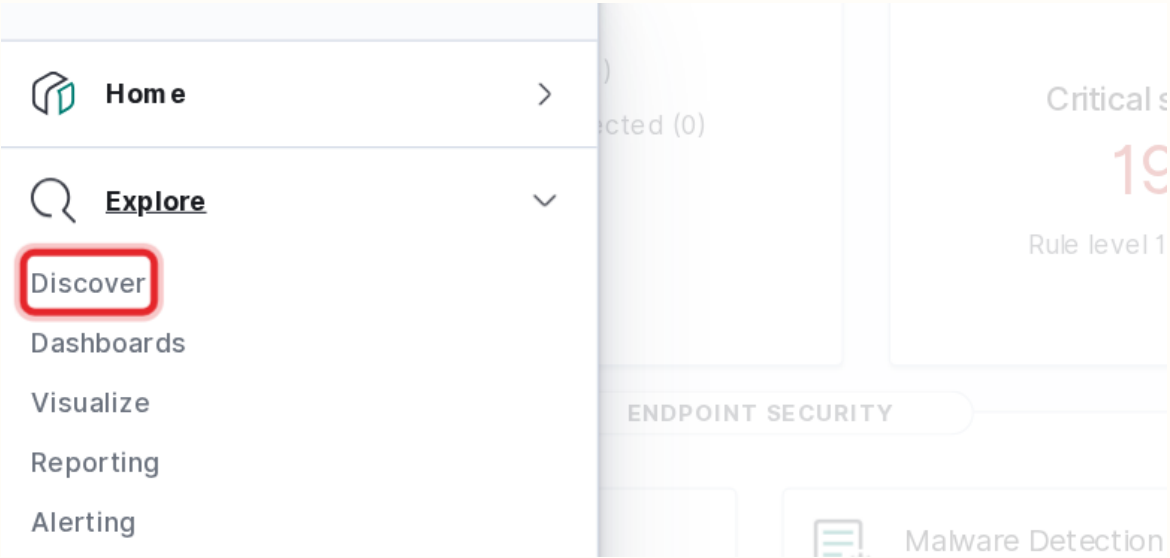
Discover로 이동하기

1. 브라우저에서 배부받은 Dashboard 주소를 열고 로그인합니다.

2. 왼쪽 위 ☰ 메뉴를 누릅니다.

3. Explore 항목을 펼치고 Discover를 누릅니다.

4. 인덱스 패턴 선택 상자에서 wazuh-alerts-*를 선택합니다.



화면 1. 기존 강의 8쪽. Explore 아래 Discover의 위치를 확인합니다.

인덱스 패턴은 어떤 기록 묶음을 검색할지 정하는 이름입니다. 이번 세 실습은 경보 인덱스에서 시작합니다. 목록에 패턴이 없으면 강사의 초기 설정을 먼저 확인합니다.

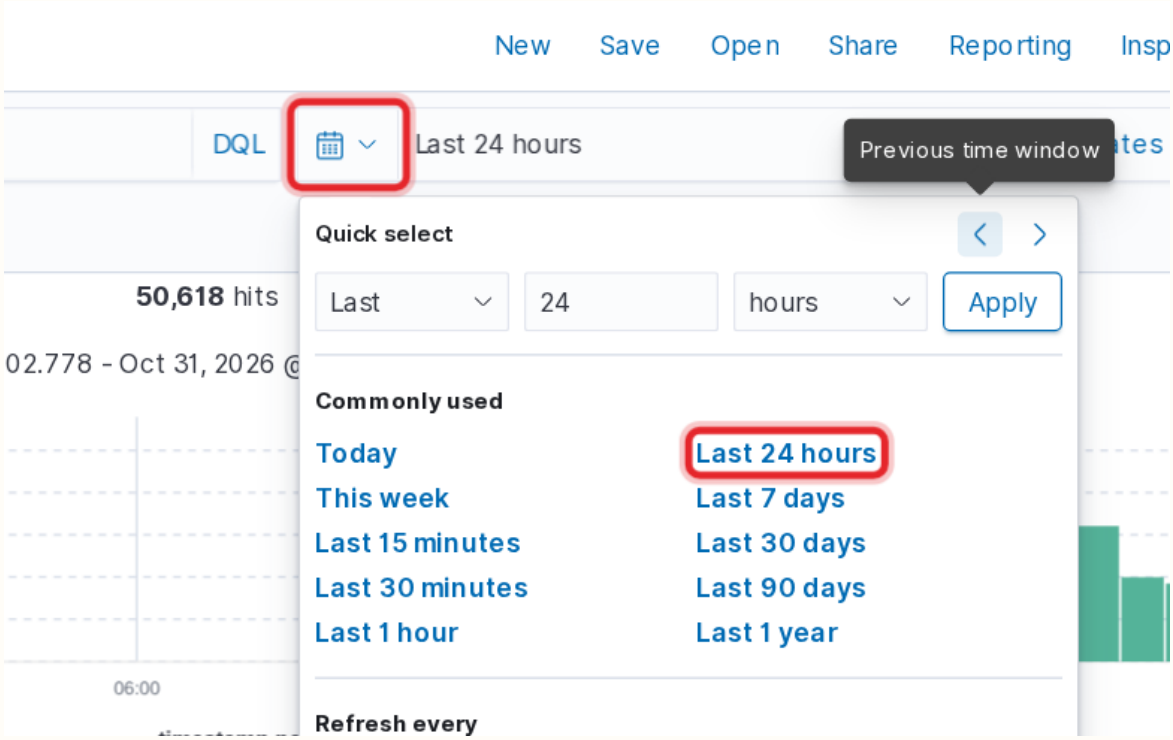
시간과 열을 먼저 맞추기

1. 검색창 옆 달력 버튼을 누릅니다.

2. 방금 실행한 실습을 찾을 때는 Last 15 minutes를 선택합니다.

3. 왼쪽 필드 검색칸에서 agent.name을 찾습니다.

4. 필드의 추가 버튼으로 결과 표에 열을 넣습니다. 같은 방법으로 rule.id와 rule.description을 추가합니다.



화면 2. 기존 강의 10쪽. 빨간 표시의 Last 24 hours는 촬영 때의 선택입니다. 이번 실습에서는 같은 목록의 Last 15 minutes를 누릅니다.

> 과거 캡처를 재현하려고 날짜를 2026-10-31로 바꾸지 않습니다. 독자가 지금 만든 이벤트는 지금 시각으로 검색해야 합니다. 수업이 길어졌으면 범위를 넓힙니다.

입력값이 질의의 의미를 바꿀 때

웹 서버는 사용자가 보낸 값을 받아 데이터를 조회합니다. 입력값을 SQL 문자열에 그대로 이어 붙이는 취약한 구현에서는, 입력한 문자가 데이터의 범위를 벗어나 SQL 구문의 일부가 될 수 있습니다.

요청의 흐름과 관찰 지점



개념 삽화 · 생성형 이미지와 별도 조판한 설명. 실제 실행 화면이 아닙니다.

```
SELECT first_name, last_name
FROM users WHERE user_id = '입력값';
```

작은따옴표는 문자열 경계를 표시합니다. UNION SELECT는 다른 조회 결과를 합치는 SQL 구문입니다. 이번에는 이런 문자열이 들어간 요청을 보내고, WAF가 차단했는지와 Wazuh에 어떤 흔적이 남았는지 확인합니다.

HTTP 200은 웹 요청에 대한 응답 코드입니다. 그것만으로 SQL 실행이나 정보 유출이 성공했다고 결론 내릴 수 없습니다.

01 정상 요청을 먼저 보내기

실행 위치: 공격자 터미널. curl은 HTTP 요청을 보내는 프로그램입니다. 이번 명령은 보안장비 설정을 바꾸는 API가 아니라, 관찰할 웹 요청 하나를 만듭니다.

```
curl --version
```

예제: examples/01-01-check.txt

버전이 출력되면 다음으로 진행합니다. command not found가 나오면 준비된 공격자 PC인지 강사에게 확인합니다.

```
read -rp 'DVWA 주소(예: http://실습호스트:8002): ' DVWA_URL
```

예제: examples/01-02-address.txt

자신의 DVWA 주소를 입력합니다. 끝에 /는 붙이지 않습니다. 이 변수는 지금 열린 터미널에서만 유지됩니다.

```
curl -sS -o /dev/null -w "HTTP %{http_code}\n" "$DVWA_URL/"
```

예제: examples/01-03-normal.txt

-o /dev/null은 본문을 버리고, -w는 상태 코드를 출력합니다. 200이나 로그인 화면으로 가는 30x라면 연결 여부를 확인할 수 있습니다. 000, 연결 거부, 이름 해석 실패라면 공격 단계로 넘어가지 않습니다.

02 공격 문자열을 한 번 보내기

같은 공격자 터미널에서 다음 명령 한 개를 실행합니다. 긴 명령은 화면에서 줄바꿈되어도 마지막 따옴표까지 한 번에 입력합니다.

```
curl -sS -o /dev/null -w "HTTP %{http_code}\n" "$DVWA_URL/
vulnerabilities/sqli/?
id=1%27%20UNION%20SELECT%20user,password%20FROM%20users--
&Submit=Submit"
```

예제: examples/01-04-request.txt

%27은 작은따옴표, %20은 공백을 URL에 표현한 것입니다. 주소 뒤에는 SQL 문법처럼 보이는 문자열이 들어 있습니다. 터미널에 표시된 응답 코드를 기록하세요.

관찰	뜻과 다음 행동
403	요청이 거부됨. WAF 기록과 함께 차단 원인을 확인
200	응답을 받음. 공격 성공 여부는 아직 모름
30x	리다이렉트됨. 로그인·경로 조건을 확인

이 실습은 SQLi 시도의 흔적과 차단 결과를 조사합니다. WAF를 끄거나 인증을 우회해야만 완주하는 절차가 아닙니다.

03 Wazuh에서 요청 찾기

분석 브라우저. Discover에서 인덱스·시간을 확인하고 다음 DQL을 입력합니다.

```
rule.id:(31103 or 31161) OR data.url:(*union* or *select* or *%27* or
*1=1* or *sleep\(*)
```

예제: examples/01-search.dql

1. 검색창에 내용을 입력하고 Enter를 누릅니다.

2. hits가 0보다 큰지 확인합니다. 캡처와 같은 건수일 필요는 없습니다.

3. 왼쪽 필드 검색을 이용해 data.srcip, data.id, data.url을 추가합니다.

4. 자신이 실행한 시각의 행을 찾고, 출발지 IP와 URL을 읽습니다.

data.srcip	data.id	data.url
192.168.0.202	403	/vulnerabilities/sqli/? 20WHEN%20%284531%3D4008 28SELECT%204008%20UNION
192.168.0.202	403	/vulnerabilities/sqli/? 78%3D2022%20AND%20%27RV
192.168.0.202	403	/vulnerabilities/sqli/? 06177%3D3686%20AND%20%2
192.168.0.202	403	/vulnerabilities/sqli/? D%20%28%27Oenv%27%3D%27
192.168.0.202	403	/vulnerabilities/sqli/? 0%27uXPX%27%3D%27uXPX&a
192.168.0.202	403	/vulnerabilities/sqli/? 2C%29%29%28.

화면 3. 기존 강의 61쪽. 자동 점검 요청이 섞인 과거 화면이라 이번 요청과 URL·건수가 다릅니다. 출발지·403 응답·공격 문자열의 위치를 확인합니다.

04 경보 한 건을 펼쳐 읽기

1. 결과 행 왼쪽의 펼치기 화살표를 누릅니다.

2. Table 탭에서 agent.name, data.srcip, data.id를 찾습니다.

3. data.url과 full_log를 찾아 입력한 공격 문자열이 있는지 확인합니다.

t	_index	wazuh-alerts-4.x-2026.10.31
t	agent.id	013
t	agent.name	web
t	data.id	403
t	data.protocol	GET
t	data.srcip	192.168.0.202

화면 4. 기존 강의 62쪽의 확대 영역. web에서 수집한 요청이며 응답 코드는 403입니다. 긴 URL은 자신의 화면에서 해당 필드를 펼쳐 읽습니다.

agent.name은 이 기록을 보낸 수집 대상입니다. 공격자 이름이 아닙니다. data.srcip는 요청 출발지이고, 웹 접근 로그에서 data.id는 여기서 응답 코드로 해석됩니다.

> 기록할 결론: “SQL 문법이 포함된 요청이 관찰되었으며 해당 요청은 403으로 거부되었다.” 이 증거만으로 데이터베이스가 침해되었다거나 다른 공격이 전혀 없었다고 단정하지 않습니다.

05 안 보일 때와 실습 마무리

1. 터미널의 응답이 000이면 먼저 웹 연결을 해결합니다.

2. Discover에서 DQL을 지우고 같은 시간에 web의 다른 경보가 보이는지 확인합니다.

3. 시간 범위와 남아 있는 필터를 확인한 뒤 검색식을 다시 입력합니다.

4. 경보가 계속 없으면 강사와 원본 접근 로그·수집 상태를 확인합니다. 화면이 비었다는 이유로 WAF를 끄지 않습니다.

강사가 원본을 확인할 때는 kt66 호스트 터미널에서 다음 명령을 한 번 실행할 수 있습니다. 포트 8002 경로의 기본 파일명이며, 이름 기반 접속이면 파일명이 달라질 수 있습니다.

```
sudo docker exec kt66-web tail -n 20 /var/log/apache2/
dvwa_port_access.log
```

예제: examples/01-troubleshoot.txt

SQLi 표지가 있어도 경보가 생성되지 않는 경우에는 수집·디코더·규칙 평가 단계를 나누어 조사합니다. 원본 한 줄의 디코더·규칙 검사는 Wazuh의 logtest로 수행할 수 있습니다. 상세 사용법은 기본 운용 장에서 다룹니다.

노트에 실행 시각, 출발지, 요청 문자열, 상태 코드, 규칙 ID, 판단을 기록합니다. 이번 요청은 설정이나 파일을 변경하지 않았으므로 별도 설정 원복은 없습니다.

실패 다음의 성공을 어떻게 읽을까

같은 출발지·계정의 시간 순서를 읽는다



개념 삽화 · 생성형 이미지와 별도 조판한 설명. 실제 실행 화면이 아닙니다.

SSH는 원격 서버에 로그인하는 통신 방법입니다. 비밀번호를 틀리게 입력하면 실패 기록이, 올바르게 입력하면 성공 기록이 남습니다. 같은 출발지에서 실패 후 성공이 이어졌다면 조사할 이유가 있지만, 사용자의 단순 오타일 수도 있습니다.

이번에는 짧은 인증 시도 패턴을 손으로 재현합니다. 공격 도구로 수십 번 시도하기 전에, 실패와 성공이 어떻게 기록되는지 직접 관찰하는 단계입니다.

> 기존 강의 18번의 Hydra 실행을 수동 인증 시도로 바꿨습니다. 자동 대입 공격의 성능이나 빈도 규칙을 검증하는 실습은 아닙니다. 이번에는 같은 계정에 실패 2회와 성공 1회만 만듭니다.

완료 기준은 실패·성공의 원본 설명을 각각 찾아 같은 출발지·계정·시간 순서로 연결하는 것입니다. 로그인 성공 경보 하나만 찾는 것으로 끝내지 않습니다.

01 대상과 시각 기록하기

실행 위치: 공격자 터미널. 강사가 지정한 Bastion 실습 계정을 사용합니다. 기본 접속 포트는 2204이며 변경된 환경은 배부값을 따릅니다.

```
read -rp 'kt66 호스트 주소: ' LAB_HOST
```

예제: examples/18-01-host.txt

```
read -rp 'SSH 실습 계정: ' LAB_USER
```

예제: examples/18-02-user.txt

```
read -rp 'SSH 포트(기본 2204): ' LAB_SSH_PORT
```

예제: examples/18-03-port.txt

```
date -Is
```

예제: examples/18-04-time.txt

비밀번호는 변수나 파일에 저장하지 않습니다. 이후 SSH가 물을 때 직접 입력합니다. 화면에는 입력한 비밀번호가 표시되지 않는 것이 정상입니다.

첫 접속이면 서버의 키 지문을 확인하는 질문이 나올 수 있습니다. 강사가 제공한 지문과 일치하는지 확인한 뒤 수락합니다. 키가 바뀌었다는 경고가 나오면 경고를 무시하는 옵션을 붙이지 말고 대상부터 확인합니다.

02 실패 두 번, 성공 한 번

키 인증으로 바로 들어가지 않도록 비밀번호 인증을 지정합니다. 아래 명령은 비밀번호를 한 번만 묻습니다.

```
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -o NumberOfPasswordPrompts=1 -p "$LAB_SSH_PORT" "$LAB_USER@$LAB_HOST"
```

예제: examples/18-05-login.txt

1. 첫 실행에서는 틀린 비밀번호를 한 번 입력합니다. Permission denied를 확인합니다.

2. 위 명령을 다시 직접 입력합니다. 두 번째에도 틀린 비밀번호를 한 번 입력합니다.

3. 위 명령을 세 번째로 실행하고 이번에는 강사가 배부한 올바른 비밀번호를 입력합니다.

4. 서버 셸에 들어오면 아래 명령을 하나씩 실행합니다.

```
hostname
```

예제: examples/18-06-hostname.txt

```
whoami
```

예제: examples/18-07-whoami.txt

```
exit
```

예제: examples/18-08-exit.txt

연결 시간 초과나 차단이 발생하면 반복 시도를 멈춥니다. 이전 실습의 누적 실패나 별도 계정 잠금 정책이 영향을 줄 수 있습니다. 자동 차단 설정을 끄지 말고 강사와 상태를 확인합니다.

03 성공 기록에서 출발지 찾기

실행 위치: 분석 브라우저. Discover에서 Last 15 minutes를 선택하고 다음 식으로 로그인 성공을 찾습니다. 내부 공격자 주소도 검색할 수 있도록 특정 대역을 제외하지 않습니다.

```
rule.id:5715
```

예제: examples/18-success.dql

1. agent.name · data.srcip · data.dstuser 열을 추가합니다.

2. 방금 성공한 시각과 계정에 해당하는 행을 찾습니다.

3. 출발지 IP를 적고 행을 펼쳐 원본 성공 메시지를 확인합니다.

t	_index	wazuh-alerts-4.x-2026.10.31
t	agent.id	005
t	agent.ip	10.20.32.254
t	agent.name	kt66-host
t	data.dstuser	ccc
t	data.srcip	192.168.0.202

화면 5. 기존 강의 228쪽. 이 캡처의 대상은 kt66-host입니다. 새 절차의 대상은 Bastion이므로 agent.name이 다릅니다. 펼친 문서에서 출발지와 계정을 읽는 참고 화면입니다.

> 공통 호스트 주소로 접속했더라도 서비스 포트에 따라 대상이 달라집니다. 책 속 에이전트 이름을 그대로 찾지 말고 자신이 실제 로그인한 대상을 확인합니다.

04 같은 출발지의 앞뒤 기록 보기

검색창을 비우고 아래 식에서 주소를 자신의 성공 기록에 나온 출발지 IP로 바꿉니다. 실제로 입력할 때 꺾쇠괄호는 남기지 않습니다.

```
rule.groups:sshd AND data.srcip:"<성공 기록의 출발지 IP>"
```

예제: examples/18-pivot.dql

1. 검색을 실행하고 Time 열 제목을 눌러 오래된 기록부터 정렬합니다.

2. 같은 계정의 실패 기록 두 건과 성공 기록을 찾습니다. 기록 수는 수집·규칙 처리에 따라 달라질 수 있습니다.

3. 실패 행과 성공 행을 각각 펼쳐 full_log의 인증 결과를 읽습니다.

4. 서로 다른 계정이나 다른 대상의 기록이 섞이면 필드 옆 필터 버튼으로 조사 대상을 좁힙니다.

시각	출발지	계정	실패 / 성공
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

실패 뒤 성공은 인증 흐름을 보여주는 증거입니다. 계정 탈취를 확정하려면 사용자 확인, 접속 위치, 이후 명령과 파일 변경 등의 추가 증거가 필요합니다.

성공은 있는데 실패가 없다면 쿼리의 규칙 ID 제한·시간 범위부터 확인합니다. 모든 기록이 없다면 강사와 Bastion의 인증 로그와 에이전트 상태를 확인합니다.

osquery로 무엇을 확인할까

이 실습 환경에는 osquery가 이미 설치되어 있습니다. kt66을 준비할 때 Linux 호스트에 osquery와 Wazuh 에이전트를 설치하고, 예약 조회와 로그 수집도 설정해 두었습니다. 독자는 설치 과정 없이 준비 상태를 확인하고 조회부터 시작합니다.

osquery는 실행 중인 프로그램, 사용자 계정, 네트워크 포트 등 운영체제 정보를 데이터베이스 테이블 형태로 제공하고 SQL 문법으로 조회하게 해 주는 도구입니다. SQL은 테이블에서 필요한 항목과 조건을 지정해 정보를 요청하는 언어입니다. 그 요청문을 쿼리라고 부릅니다.

테이블은 같은 종류의 정보를 모은 표입니다. 가로 한 줄인 행은 조사 대상 하나, 세로 항목인 열은 그 대상의 속성입니다. 예를 들어 실행 중인 프로그램을 프로세스라고 하며, processes 테이블의 한 행은 프로세스 하나를 나타냅니다.

테이블 이름	조회할 정보
processes	실행 중인 프로세스의 번호(pid), 이름(name), 실행 파일 경로(path)
listening_ports	연결을 기다리는 포트 번호(port), 주소(address), 프로세스 번호(pid)
authorized_keys	SSH 로그인을 허용할 공개키의 파일 위치(key_file), 주석(comment)

이번에 쓰는 테이블은 조회 시점에 운영체제나 관련 파일에서 정보를 읽어 결과를 구성합니다. 이를 가상 테이블이라고 합니다. 운영체제 정보 전체를 미리 데이터베이스에 옮겨 관리한다는 뜻은 아닙니다. 예약 조회에서는 변화를 비교하기 위해 이전 결과를 따로 보관합니다.

> 실행 장소는 kt66이 설치된 Linux 호스트입니다. 개인 노트북이나 Bastion 컨테이너에서 실행하면 조사 대상이 달라집니다. 앞의 SSH 로그가 “어떤 접속이 있었는가”를 보여 줬다면, 이번 조회는 “지금 어떤 프로그램·포트·키가 있는가”를 확인하는 데 쓰입니다.

이 장은 설치와 호스트 센서 준비가 끝난 환경을 전제로 합니다. 다음 쪽의 확인에 실패하면 도구를 임의로 설치하지 말고 접속한 호스트와 kt66 준비 상태부터 확인합니다.

01 설치된 도구와 서비스 확인하기

실행 위치: kt66 호스트 터미널 A. 설치 작업은 하지 않습니다. 먼저 접속한 호스트와 이미 준비된 서비스가 동작하는지 확인합니다. 서비스는 백그라운드에서 계속 실행되는 프로그램입니다. osqueryd는 예약 조회를, wazuh-agent는 수집·전송을 담당합니다.

```
hostname
```

예제: examples/62-01-hostname.txt

```
systemctl is-active osqueryd
```

예제: examples/62-02-osquery-status.txt

```
systemctl is-active wazuh-agent
```

예제: examples/62-03-agent-status.txt

```
command -v nc
```

예제: examples/62-04-nc.txt

systemctl is-active는 서비스의 실행 상태를, command -v는 명령의 위치를 확인합니다. 두 서비스는 active, nc는 실행 파일 경로가 나와야 합니다. inactive·failed·not found 또는 경로가 없는 경우에는 대상 호스트와 kt66 준비 상태를 강사와 확인합니다. 이어서 cat으로 예약 조회 설정 파일을 읽습니다.

```
cat /etc/osquery/osquery.conf
```

예제: examples/62-05-config.txt

kt66_listening_ports·kt66_authorized_keys 항목과 interval 값을 찾습니다. 해당 항목이 없으면 강사에게 kt66 센서 준비를 요청합니다. 여기서 설정 전체를 복사해 덮어쓰지 않습니다.

interval은 조회 간격(초)입니다. 제공 설정은 두 항목 모두 60입니다. 설정을 읽는 단계이므로 파일을 바꾸지 않습니다.

02 SQL로 호스트 정보 읽기

실행 위치: kt66 호스트 터미널 A. osquery에는 사람이 SQL을 입력하는 osqueryi와 예약 조회를 수행하는 osqueryd가 있습니다. 먼저 설치된 osqueryi의 위치를 확인합니다.

```
command -v osqueryi
```

예제: examples/62-intro-01-location.txt

실행 파일 경로가 나오면 다음 명령으로 조회 창을 엽니다. 아무것도 나오지 않으면 접속한 호스트와 osquery 설치 상태를 확인합니다.

```
osqueryi
```

예제: examples/62-intro-02-open.txt

osquery>가 보이면 아래 SQL을 입력하고 Enter를 누릅니다. 프롬프트 글자는 입력하지 않습니다.

```
SELECT pid, name, path FROM processes WHERE pid = 1;
```

예제: examples/62-intro-03-process.sql

결과에서 pid가 1인 행을 찾습니다. 다음 쪽에서 SQL의 각 부분과 결과 읽는 법을 확인합니다.

02 SQL의 구조와 결과 읽기

구문	뜻
SELECT pid, name, path	프로세스 번호·이름·실행 파일 경로 열을 가져옵니다.
FROM processes	프로세스 정보를 제공하는 테이블에서 찾습니다.
WHERE pid = 1	프로세스 번호가 1인 행만 고릅니다.
;	SQL 문장이 끝났음을 표시합니다.

결과에서 pid가 1인 행과 name·path 값을 읽습니다. 이름과 경로는 환경마다 다릅니다. ...>가 나오면 입력이 끝나지 않은 상태이므로 빠진 세미콜론을 확인합니다. 문법 오류가 나면 쉼표·철자·끝의 세미콜론을 예제와 비교합니다.

```
.exit
```

예제: examples/62-intro-04-exit.txt

.exit는 조회 창을 닫습니다. 일반 셸 프롬프트로 돌아온 것을 확인합니다. 이 SQL은 호스트 정보를 읽으며, 프로세스를 종료하거나 설정을 바꾸지 않습니다. osqueryi에서 조회한 결과가 Wazuh에 자동 전송되는 것은 아닙니다.

03 예약 조회와 Wazuh 수집

호스트의 변화가 분석 화면에 나타나기까지



호스트의 변화
포트·인증 키 추가
→
osqueryd 예약 조회
이전 결과와 비교
→
Wazuh 분석
추가된 항목 확인

개념 삽화 · 생성형 이미지와 별도 조판한 설명. 실제 실행 화면이 아닙니다.

앞쪽은 사람이 osqueryi에 직접 SQL을 입력했습니다. 이제는 kt66이 미리 설정해 둔 osqueryd의 예약 조회가 실습으로 만든 변화를 발견하는 과정을 관찰합니다.

1. osqueryd가 /etc/osquery/osquery.conf의 SQL을 정해진 간격으로 실행합니다.

2. 이전 결과와 비교해 새 행은 added, 사라진 행은 removed로 기록합니다. 이 비교 기록을 차이 로그라고 합니다.

3. 결과는 /var/log/osquery/osqueryd.results.log에 JSON 형식으로 남습니다. JSON은 항목 이름과 값을 짝지어 표현하는 데이터 형식입니다.

4. 호스트의 Wazuh 에이전트가 이 파일을 읽어 매니저로 전송합니다. 매니저의 규칙 평가로 생성된 경보가 인덱서에 저장되면 Dashboard에서 검색할 수 있습니다.

03 추가 기록을 해석하는 기준

이 환경에서 osqueryd는 별도 서비스로 실행됩니다. Wazuh는 그 결과를 수집합니다. 같은 “조회”라도 osquery의 SQL은 호스트 정보를 읽고, Discover의 DQL은 수집되어 인덱서에 저장된 기록을 찾습니다.

> 처음 실행할 때는 비교할 이전 결과가 없어 기존 항목도 added로 기록될 수 있습니다. 시작 뒤 1~2분 기다려 기준 결과가 만들어진 다음 실습을 진행합니다. 60초 간격에는 분산 지연도 있어 변화를 만든 뒤 1~2분 기다립니다.

이번 포트·키 조회 사이에 나타났다가 사라진 상태는 관찰하지 못할 수 있습니다. added는 새 결과 행이라는 뜻이지 악성 행위의 판정이 아닙니다. 포트와 공개키를 하나씩 추가해 관찰한 뒤 직접 원복하겠습니다.

두 번의 조회 사이에는 빈 구간이 있습니다

아래는 원리를 설명하기 위한 가상 시각이며 실행 결과가 아닙니다. 실제 조회 간격은 설정과 분산 지연에 따라 달라집니다.

가상 시각	호스트에서 일어난 일
10:00:00	예약 조회: 31338 포트가 없음
10:00:10	31338 포트를 열음
10:00:20	31338 포트를 닫음
10:01:00	다음 조회: 31338 포트가 없음

두 조회 결과가 같으면 이 포트가 잠깐 열렸던 일을 차이 로그로 알 수 없습니다. 다음 실습에서는 nc를 실행한 창을 유지하고, 기록을 확인한 뒤 종료합니다. 과거 행위는 앞서 배운 이벤트 로그와 함께 조사해야 합니다.

04 리스닝 포트 만들기

터미널 A에서 해당 포트가 이미 사용 중인지 먼저 확인합니다.

```
ss -ltn 'sport = :31338'
```

예제: examples/62-06-port-before.txt

제목 행 아래에 리스닝 항목이 없어야 합니다. 사용 중이면 기존 프로세스를 종료하지 말고 다른 실습과 겹치는지 확인합니다.

```
nc -l 0.0.0.0 31338
```

예제: examples/62-07-listen.txt

프롬프트가 돌아오지 않는 것은 정상입니다. 이 창을 그대로 둡니다. 다른 터미널 B를 열어 같은 호스트에 접속한 뒤 아래 명령을 입력합니다.

```
ss -ltn 'sport = :31338'
```

예제: examples/62-08-port-after.txt

이번에는 LISTEN과 31338이 보여야 합니다. 이는 nc가 연결을 기다리는 상태라는 뜻이며, 악성코드나 원격 셸의 존재를 입증하지는 않습니다.

> kt66 예약 쿼리는 127.0.0.1 등의 로컬 전용 주소를 제외합니다. 이 실습에서는 지정된 격리 호스트에서 0.0.0.0으로 듣도록 합니다. 외부 연결을 시도할 필요는 없습니다.

05 예약 쿼리 결과 찾기

1~2분 뒤 분석 브라우저의 Discover에서 시간을 Last 15 minutes로 맞추고 검색합니다.

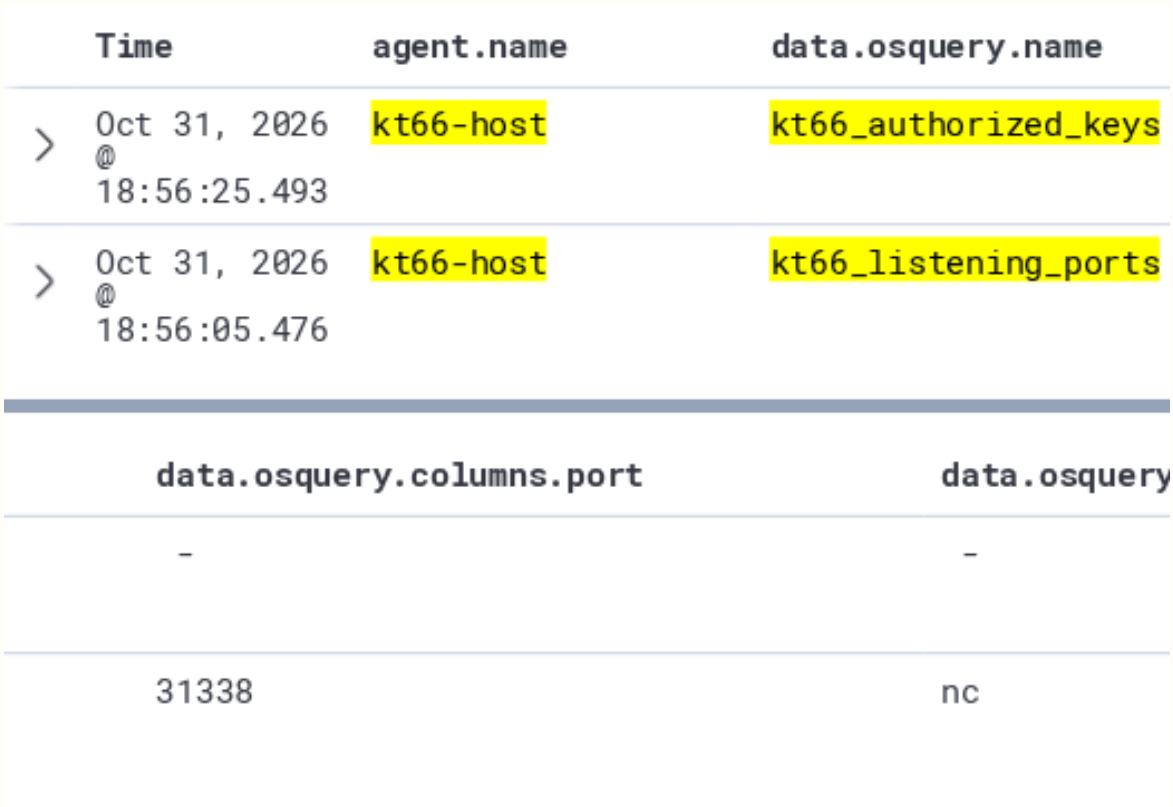
```
rule.groups:osquery AND agent.name:kt66-host AND
data.osquery.name:kt66_listening_ports AND data.osquery.action:added
```

예제: examples/62-ports.dql

1. data.osquery.name, data.osquery.action, data.osquery.columns.port 열을 추가합니다.

2. 31338이 있는 행을 펼칩니다.

3. data.osquery.columns.name에서 nc를 확인합니다.



화면 6. 기존 강의 633쪽. 아래는 nc의 31338 포트, 위는 다음에 살펴볼 인증 키입니다.

06 실습용 SSH 키 하나 추가하기

실행 위치: 호스트 터미널 B. 자신의 실습 계정에서만 진행합니다. 개인키는 외부로 보내지 않습니다. 먼저 같은 표지가 있는 키가 없는지 확인합니다.

```
grep -n 'book62-demo' ~/.ssh/authorized_keys
```

예제: examples/62-09-existing.txt

기존 항목이 나오면 중복 실행하지 않습니다. 파일이 없다는 메시지는 아직 키 파일이 없다는 뜻입니다. 이어서 아래 명령을 한 줄씩 실행합니다.

```
BOOK_KEY_DIR=$(mktemp -d)
```

예제: examples/62-10-temp.txt

```
ssh-keygen -t ed25519 -f "$BOOK_KEY_DIR/demo" -N "" -C book62-demo
```

예제: examples/62-11-keygen.txt

```
mkdir -p ~/.ssh
```

예제: examples/62-12-mkdir.txt

```
chmod 700 ~/.ssh
```

예제: examples/62-13-mode.txt

ssh-keygen은 실제 유효한 키를 만듭니다. 샘플의 AAAA... 같은 잘린 문자열을 넣지 않습니다. 다음 페이지에서 공개키만 등록합니다.

07 공개키 등록 후 변화 확인하기

터미널 B에서 먼저 공개키 내용을 읽고 마지막 주석이 book62-demo인지 확인합니다.

```
cat "$BOOK_KEY_DIR/demo.pub"
```

예제: examples/62-14-public.txt

```
printf '\n' >> ~/.ssh/authorized_keys
```

예제: examples/62-15-newline.txt

```
cat "$BOOK_KEY_DIR/demo.pub" >> ~/.ssh/authorized_keys
```

예제: examples/62-16-append.txt

```
chmod 600 ~/.ssh/authorized_keys
```

예제: examples/62-17-mode.txt

1~2분 뒤 Discover의 검색창을 다음 식으로 바꿉니다.

```
rule.groups:osquery AND agent.name:kt66-host AND
data.osquery.name:kt66_authorized_keys AND data.osquery.action:added
```

예제: examples/62-keys.dql

해당 시각의 문서를 펼쳐 data.osquery.columns.comment가 book62-demo인지 확인합니다. username과 key_file도 읽습니다. added 한 단어만 보고 공격자 키라고 판정하지 않습니다.

08 공개키 삭제 명령 이해하기

호스트 터미널 B, 공개키를 등록한 계정. sed(stream editor)는 텍스트를 줄 단위로 읽고 찾기·바꾸기·삭제하는 도구입니다.

기본 구문: sed 옵션 '편집 규칙' 파일

구성 요소	뜻
-i	편집 결과를 원본 파일에 저장합니다. 생략하면 결과만 화면에 출력하고 파일은 바꾸지 않습니다.
'/ book62-demo$/d'	작은따옴표는 편집 규칙을 셸이 임의로 해석하지 않고 sed에 전달하게 합니다.
/ book62-demo$/	두 / 사이가 찾을 패턴입니다. 맨 앞은 공백 한 칸이며, $는 줄 끝을 뜻합니다.
d	패턴에 맞는 줄 전체를 삭제합니다. 키의 주석 글자만 지우는 것이 아닙니다.
~/.ssh/authorized_keys	~는 현재 계정의 홈 디렉토리입니다. 이 파일에는 SSH 로그인을 허용할 공개키가 저장됩니다.

먼저 삭제할 줄을 확인합니다. grep은 패턴이 있는 줄을 찾고, -n은 줄 번호를 붙입니다. 결과가 이번에 등록한 키 한 줄인지 확인하세요. 없거나 여러 줄이면 삭제 전에 등록 상태를 다시 확인합니다.

```
grep -n 'book62-demo' ~/.ssh/authorized_keys
```

예제: examples/62-09-existing.txt

줄 끝이 공백과 book62-demo인 모든 행을 삭제합니다.

```
sed -i '/ book62-demo$/d' ~/.ssh/authorized_keys
```

예제: examples/62-19-key-cleanup.txt

09 삭제 확인과 나머지 원복

실행 위치: 호스트 터미널 B. 공개키 행을 삭제한 뒤 같은 검색을 다시 실행합니다.

```
grep -n 'book62-demo' ~/.ssh/authorized_keys
```

예제: examples/62-20-check-cleanup.txt

아무 줄도 출력되지 않고 입력 프롬프트로 돌아오면 표지가 있는 줄이 사라진 것입니다. 파일이 없거나 권한이 없다는 오류는 성공이 아닙니다. 행이 남으면 바로 삭제를 반복하지 말고 줄 끝의 주석과 공백을 확인합니다.

터미널 A로 돌아가 Ctrl+C를 눌러 nc를 종료합니다. 터미널 B에서 포트 31338의 리스닝 항목이 사라졌는지 확인합니다.

```
ss -ltn 'sport = :31338'
```

예제: examples/62-18-port-cleanup.txt

같은 터미널에서 임시 디렉토리 경로를 확인하고, 이번에 생성한 두 키 파일만 삭제합니다. rm은 파일을, rmdir은 빈 디렉토리를 삭제합니다. -- 뒤의 값은 옵션이 아닌 경로로 취급합니다.

```
printf "%s\n" "$BOOK_KEY_DIR"
```

예제: examples/62-21-path.txt

```
rm -- "$BOOK_KEY_DIR/demo" "$BOOK_KEY_DIR/demo.pub"
```

예제: examples/62-22-remove.txt

```
rmdir -- "$BOOK_KEY_DIR"
```

예제: examples/62-23-rmdir.txt

> 터미널을 닫아 변수가 사라졌다면 마지막 삭제 명령을 그대로 실행하지 말고, 처음 생성했던 임시 디렉토리의 실제 위치를 확인합니다.

10 결과가 없을 때 순서대로 확인하기

1. nc가 계속 실행 중인지 터미널 A와 ss 결과를 확인합니다.

2. 예약 간격을 기다렸는지 확인합니다. 즉시 검색만 반복하지 않습니다.

3. 호스트 이름·에이전트 이름·시간 범위·DQL 필터를 확인합니다.

4. 터미널 B에서 결과 로그를 읽어, Wazuh 이전 단계에 이벤트가 있는지 확인합니다.

```
sudo tail -n 20 /var/log/osquery/osqueryd.results.log
```

예제: examples/62-24-results.txt

원본에 kt66_listening_ports 또는 kt66_authorized_keys와 이번 값이 있으면 osquery의 생성 단계는 확인한 것입니다. 원본은 있는데 Dashboard에 없다면 Wazuh 수집 경로를 조사합니다. 원본에도 없으면 예약 설정·서비스·조회 조건부터 확인합니다.

원복 뒤에는 아래 식으로 삭제 변화도 확인해 봅니다. 삭제 경보가 없다는 이유로 원복이 실패했다고 단정하지 않고, ss와 authorized_keys의 실제 상태를 먼저 봅니다.

```
rule.groups:osquery AND agent.name:kt66-host AND
data.osquery.action:removed
```

예제: examples/62-removed.dql

노트에는 포트·프로세스·키 주석, 추가 시각, 발견 시각, 원복 확인 결과를 남깁니다.

세 실습을 마친 뒤

증거와 판단을 구분해서 쓰기

실습	직접 본 증거	추가로 알아야 할 것
SQL 인젝션	공격 문자열·응답 코드·경보	SQL 실행과 실제 유출 여부
SSH	실패 후 성공·출발지·계정	정상 사용자 여부·로그인 이후 행위
osquery	포트·키 상태 변화	변경 주체·승인된 변경 여부

이번 실습에서는 Indexer API로 답을 가져오지 않았습니다. 직접 만든 행위를 검색창에서 찾고, 열을 추가하고, 원본 문서를 펼치며 증거를 읽었습니다. 다음 실습에서도 같은 순서를 유지합니다.

제출물: 세 실습 각각 실행 시각, 대상, 검색식, 확대 결과 화면, 관찰 사실, 판단의 한계, 원복 결과를 한 장에 정리하세요.

자료와 화면 출처

강의 레포 hack_ir_with_wazuh: 36897d0. kt66 구성: 63d4180. 실제 캡처의 촬영 당시 날짜는 2026-10-31이며, 그림·절차는 이 시범본에 맞게 별도로 작성했습니다.

Wazuh 공식 문서: Testing decoders and rules — 원본 한 줄의 디코더·규칙 평가를 검사하는 방법. 현재 문서와 실습 버전의 차이는 확인이 필요합니다.

osquery: SQL Introduction · Using osqueryi · Logging. 가상 테이블·직접 조회·차이 로그의 설명을 대조했습니다.